

דיוק פעולות:

עקב הייצוג הסופי של הממשיים, גם בביצוע פעולות ביניהם יש דיוק סופי.

כפל: נניח $x = \tilde{x} + \delta x$ כאשר x, y הם הייצוג במחשב של המספרים הממשיים $\tilde{x}, \tilde{y}$. בנוסף, נסמן את $y = \tilde{y} + \delta y$

שגיאת הקיצוץ של פעולת כפל ב- ε . נבחן את השגיאה היחסית של הכפל $z = x \cdot y$:

$$\frac{\delta z}{\tilde{z}} = \frac{(\tilde{x} + \delta x) \cdot (\tilde{y} + \delta y) + \varepsilon - \tilde{x} \cdot \tilde{y}}{\tilde{x} \cdot \tilde{y}} = \frac{\tilde{x} \cdot \tilde{y} + \tilde{y} \cdot \delta x + \tilde{x} \cdot \delta y + \delta x \cdot \delta y + \varepsilon - \tilde{x} \cdot \tilde{y}}{\tilde{x} \cdot \tilde{y}} \approx \frac{\delta x}{\tilde{x}} + \frac{\delta y}{\tilde{y}} + \frac{\varepsilon}{\tilde{z}}$$

כלומר, השגיאה היחסית של z היא השגיאה המצטברת של x, y ופעולת הכפל ε .

חילוק:

$$z = \frac{y}{x} = \frac{\tilde{y} + \delta y}{\tilde{x} + \delta x} + \varepsilon = \frac{\tilde{y} + \delta y}{\tilde{x} \cdot (1 + \delta x / \tilde{x})} + \varepsilon \approx \frac{(\tilde{y} + \delta y) \cdot (1 - \delta x / \tilde{x})}{\tilde{x}} + \varepsilon$$

$$\Rightarrow \frac{z - \tilde{z}}{\tilde{z}} = \frac{\frac{\tilde{y}}{\tilde{x}} + \frac{\delta y}{\tilde{x}} - \frac{\tilde{y} \cdot \delta x}{\tilde{x}^2} - \frac{\delta y \cdot \delta x}{\tilde{x}^2} - \tilde{z} + \varepsilon}{\tilde{z}} = \frac{\delta y}{\tilde{y}} - \frac{\delta x}{\tilde{x}} + \frac{\varepsilon}{\tilde{z}}$$

כלומר, קיבלנו תוצאה דומה לתוצאת הכפל, שגיאה מצטברת.

$$\frac{z - \tilde{z}}{\tilde{z}} = \frac{\delta x + \delta y + \varepsilon}{\tilde{x} + \tilde{y}} \quad \text{חיבור וחסור: השגיאה היחסית היא}$$

כאן עלולה להיווצר בעיה מכיוון שהמכנה יכול להיות קטן מהמונה.

דוגמה לבעייתיות - משוואה ריבועית: $x^2 - 2 \cdot b \cdot x + c = 0$

$$(*) \quad x_{1,2} = b \pm \sqrt{b^2 - c} \quad \text{הפתרונות של המשוואה ניתנים על ידי הנוסחה המוכרת:}$$

אם b גדול הרבה יותר מ- c (למשל $b = 10^6$ $c = -10^{-6}$) הרי שאם נשתמש ב- $(*)$ נקבל

$x_1 = 2 \cdot b = 2 \cdot 10^6$ ו- $x_2 = 0$. אם נציב את הפתרונות שקיבלנו במשוואה נראה שהפתרון הראשון אכן

מהווה קירוב טוב לפתרון הנכון (לפחות באופן יחסי) בעוד שהפתרון השני כמובן שגוי.

ניתן לקבל את הפתרון השני אם נשתמש בנוסחה מוכרת פחות לפתרונות משוואה ריבועית:

$$(+)\quad x_{1,2} = \frac{c}{b \mp \sqrt{b^2 - c}}$$

לפי נוסחה זו (+) נקבל $x_1 = INF$ ו- $x_2 = \frac{c}{2 \cdot b} = -5 \cdot 10^{-13}$.

כלומר, כשהטווח הדינמי של המשתנים הוא גדול, יש להתחשב בדיוק הסופי של המחשב כדי לא לקבל תוצאות שגויות או טעויות בזמן ריצה ולהשתמש באלגוריתם הלוקח זאת בחשבון (במקרה זה, בנוסחה המתאימה).

דיוק

מקור נוסף להבדל בין התוצאה המדויקת לתוצאה המחושבת על-ידי תוכנית מחשב הוא הדיוק הנקרא גם שגיאת קיטוע (truncation error). שגיאות אלו הן תוצאה של דיוק סופי של האלגוריתם של החישוב, והן יתבטאו גם במחשב עם דיוק אין סופי. לדוגמא, בחישוב נומרי של אינטגרל סופי, מחשבים את ערך הפונקציה במספר סופי של נקודות. וכך, נוצר הפרש בין התוצאה המדויקת למחושבת. דוגמה נוספת היא שימוש בפיתוח לטור מתכנס לצורך הערכה של גודל מבוקש. במקרה זה, ככל שנתחשב ביותר אברים בטור כך ערכו של הגודל יהיה מדויק יותר, וההפרש בין הערך האמיתי לגודל שחושב שווה לסכום האברים שהוזנחו ומהווה את שגיאת הקיטוע.

יציבות

בעיה מיוחדת מתעוררת כאשר יש אינטראקציה חזקה בין שגיאות העיגול במחשב והדיוק של האלגוריתם. מצב זה מתעורר כשהאלגוריתם אינו יציב. כלומר, במצב בו שגיאה הנובעת מעיגול המחשב מוגברת תוך כדי החישוב עד שהיא הופכת להיות דומיננטית בחישוב.

דוגמה לאלגוריתם לא יציב, חישוב חזקות שלמות של "ממוצע הזהב" (golden mean), שהוא

$$\phi = \frac{\sqrt{5}-1}{2} \approx 0.61803398. \text{ מסתבר שמספר זה מקיים את תנאי הרקורסיה } \phi^{n+1} = \phi^{n-1} - \phi^n. \text{ כך}$$

ניתן להשתמש בנוסחה זו ובערכי ההתחלה כדי לחשב בצורה מהירה את החזקות של ϕ .

בחישוב במחשב עם מספרים ממשיים המיוצגים על-ידי ארבעה בתים, מסתבר שמקבלים תוצאות שגויות לגמרי

עבור $n = 20$, כאשר הערך המדויק הוא רק $\sim \frac{1}{2} \cdot 10^{-4}$, למרות שהדיוק היחסי של מספרים ממשיים עם

ארבעה בתים הוא 10^{-7} ולכאורה היה אפשר לקבל הערכה טובה למדי של ϕ^{20} .

הסיבה לכך היא שלמשוואה פתרון נוסף שהוא $\frac{-\sqrt{5}-1}{2}$. מכיוון שמשוואת הרקורסיה היא ליניארית ומכיוון

שערכו המוחלט של הפתרון השלילי גדול מ-1, הרי שרכיב קטן של פתרון זה שייכנס עקב שגיאות עיגול יגדל אקספוננציאלית. כלומר, אם נסמן את שתי הפתרונות: $\phi_+ \approx 0.618$ ו- $\phi_- \approx -1.618$ הרי שבגלל הדיוק

הסופי, הערך של האיטרציה הראשונה במחשב הוא $\phi^1 = \phi_+ + \alpha \cdot \phi_-$ כאשר $\alpha \approx 10^{-7}$ (שכן הצגה של

מספר ממשי עם 4 בתים מדויקת עד כדי 10^{-7} וניתן להביע את ההפרש בין הערך המדויק לערך במחשב כמכפלה

של הפתרון השלילי במספר שהוא מסדר גודל של 10^{-7}). אם נניח שזו השגיאה היחידה הרי שבגלל הליניאריות

של משוואת הריקורסיה נקבל במהלך חישוב החזקות $\phi^n = \phi_+^n + \alpha \cdot \phi_-^n$ ומכאן נקבל שהשגיאה היחסית היא:

$$\frac{\phi^n - \phi_+^n}{\phi_+^n} = \frac{\alpha \cdot \phi_-^n}{\phi_+^n}$$

מכאן ברור שבשגיאה גדלה מהר שהרי ϕ_-^n גדל מהר ואילו ϕ_+^n קטן מהר.

נבחן זאת על ידי פונקציית הפייתון stab המקבלת כפרמטרים את n – החזקה המרבית לחישוב ואת type שצריך להיות 'float32' (עבור הצגת ממשיים עם ארבעה בתים או 32 סיביות) או 'float64' (עבור הצגת ממשיים עם 8 בתים או 64 סיביות). פונקציית הפייתון משתמשת ב-numpy, ומחזירה מערך דו-ממדי בו לכל i מוצב ϕ^i שחושב באופן ישיר וכן הערך שהתקבל מנוסחת הריקורסיה והשגיאה היחסית. במהלך פעולת הפונקציה היא גם מדפיסה גדלים אלו.

```
In [1]: import numpy as np

def stab(n,type):
    a = np.zeros([n,3],dtype=type)
    a[0,0] = 1.0
    a[1,0] = (np.sqrt(5) - 1) / 2
    phi_1 = a[1,0]
    phi_im = a[0,0]
    phi_i = phi_1

    for i in range(2,n):
        phi_ip = phi_im - phi_i
        phi_im = phi_i
        phi_i = phi_ip
        a[i,:] = [phi_i, phi_1**i, 1-phi_i/phi_1**i]
        print('%3d %12.4e %12.4e %12.4e ' % (i, phi_1**i, phi_i, 1-phi_i/phi_1**i))

    return a
```

בקריאה לפונקציה עם n=25 ו-type='float32' מתקבלת ההדפסה הבאה:

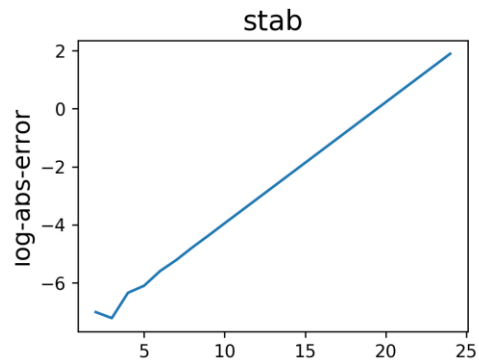
```
In [2]: golden = stab(25, 'float32')
```

2	3.8197e-01	3.8197e-01	9.6096e-08
3	2.3607e-01	2.3607e-01	-5.9391e-08
4	1.4590e-01	1.4590e-01	4.4378e-07
5	9.0170e-02	9.0170e-02	-7.7744e-07
6	5.5728e-02	5.5728e-02	2.5158e-06
7	3.4442e-02	3.4442e-02	-6.0100e-06
8	2.1286e-02	2.1286e-02	1.6407e-05
9	1.3156e-02	1.3156e-02	-4.2185e-05
10	8.1306e-03	8.1297e-03	1.1131e-04
11	5.0250e-03	5.0265e-03	-2.9044e-04
12	3.1056e-03	3.1033e-03	7.6145e-04
13	1.9194e-03	1.9232e-03	-1.9924e-03
14	1.1862e-03	1.1801e-03	5.2173e-03
15	7.3314e-04	7.4315e-04	-1.3658e-02
16	4.5310e-04	4.3690e-04	3.5758e-02
17	2.8003e-04	3.0625e-04	-9.3614e-02
18	1.7307e-04	1.3065e-04	2.4509e-01
19	1.0696e-04	1.7560e-04	-6.4164e-01
20	6.6107e-05	-4.4942e-05	1.6798e+00

כלומר, השגיאה היחסית "מתפוצצת" וערכה המוחלט גדל בערך בפקטור כפלי הגדול מ-2 בכל איטרציה או חזקה. עוד רואים שהשגיאה משנה סימן בכל פעם.

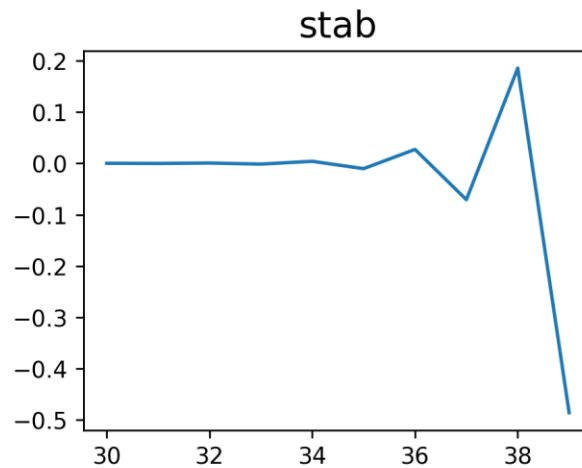
ניתן לראות את הגידול המהיר של השגיאה היחסית בגרף חצי לוגריתמי של הערך המוחלט של השגיאה, כפי שמוצג להלן על ידי שימוש בחבילת matplotlib בתוך מחברת jupyter.

```
In [23]: %matplotlib notebook
from matplotlib import pyplot as plt
import numpy as np
plt.figure(figsize=(4,3))
plt.plot(range(2,25),np.log10(abs(golden[2:25,2])))
plt.title('stab',fontsize=16)
plt.xlabel('n',fontsize=12)
plt.ylabel('log-abs-error',fontsize=14)
plt.show()
```



גם כשעובדים עם float64 החישוב בלתי יציב, אלא ש"מרוויחים" כ-20 חזקות. התנהגות זאת מוצגת במחברת jupyter הבאה בה רואים את השגיאה היחסית כתלות בחזקה. בהצגה זו ניכר היפוך הסימן של השגיאה בכל איטרציה.

```
In [28]: ▶ %matplotlib notebook
from matplotlib import pyplot as plt
import numpy as np
plt.figure(figsize=(4,3))
plt.plot(range(30,40),stab(41,'float64')[30:40,2])
plt.title('stab',fontsize=16)
plt.xlabel('n',fontsize=14)
plt.ylabel('error',fontsize=14)
plt.show()
```



2	3.8197e-01	3.8197e-01	3.3307e-16
3	2.3607e-01	2.3607e-01	-2.2204e-16
4	1.4590e-01	1.4590e-01	1.5543e-15
5	9.0170e-02	9.0170e-02	-2.6645e-15
6	5.5728e-02	5.5728e-02	8.3267e-15
7	3.4442e-02	3.4442e-02	-1.9984e-14
8	2.1286e-02	2.1286e-02	5.4290e-14
9	1.3156e-02	1.3156e-02	-1.3967e-13
10	8.1306e-03	8.1306e-03	3.6826e-13
11	5.0250e-03	5.0250e-03	-9.6101e-13
12	3.1056e-03	3.1056e-03	2.5198e-12
13	1.9194e-03	1.9194e-03	-6.5932e-12
14	1.1862e-03	1.1862e-03	1.7265e-11
15	7.3314e-04	7.3314e-04	-4.5196e-11
16	4.5310e-04	4.5310e-04	1.1833e-10
17	2.8003e-04	2.8003e-04	-3.0979e-10
18	1.7307e-04	1.7307e-04	8.1104e-10
19	1.0696e-04	1.0696e-04	-2.1233e-09
20	6.6107e-05	6.6107e-05	5.5589e-09
21	4.0856e-05	4.0856e-05	-1.4553e-08
22	2.5251e-05	2.5251e-05	3.8101e-08
23	1.5606e-05	1.5606e-05	-9.9751e-08
24	9.6449e-06	9.6449e-06	2.6115e-07
25	5.9609e-06	5.9609e-06	-6.8370e-07

ניתן להשתמש בנוסחת הרקורסיה לחישוב החזקות של ממוצע הזהב בדרך אחרת. בדרך זו "הופכים" את הנוסחה: $\phi^{n-1} = \phi^n + \phi^{n+1}$. כלומר, ניתן "ללכת אחורה" ולחשב את החזקות של ממוצע הזהב מ- n ל-1 או 0 אם יודעים את הערך עבור n ועבור $n+1$. למעשה, אין צורך לדעת את הערך עבור n אלא אפשר להניח שהערך הוא סתמי (למשל 0.1) והערך עבור $n+1$ הוא $\phi^{n+1} = 0.1 \phi^1$.

ניתן להשתמש בנוסחת הרקורסיה אחורנית ולחשב את החזקות היורדות, עד $n=0$. מכיוון שהתחלנו מערך סתמי הרי שלא נקבל $\phi^0 = 1$, אך בגלל ליניאריות המשוואה, אם נחלק את כל החזקות שקיבלנו בערך שקיבלנו עבור $n=0$, נקבל את התוצאה הנכונה.